Compiler Design Laboratory Assignment -II

Design and Implementation of a Lexical Analyzer for a Toy Language

Objective

The objective of this experiment is to design and implement a **Lexical Analyzer** for a given toy programming language. Through this assignment, the student will:

- Understand the concepts of lexemes, tokens, and patterns.
- Design transition diagrams for token recognition.
- Implement token recognition using program code.
- Generate tokens from a source program.
- Construct and maintain a symbol table.

Given Language Specification

1. Alphabet Set

- Letters: `A--Z, a--z`
- Digits: `0--9`
- Special symbols: `; , ( ) { } [ ]`

2. Keywords

`if, else, while, int, float, return, begin, end, print`

3. Operators

`+ - * / =`

4. Relational Operators

`< > <= >= == !=`

5. Constants

- Integer constants: `0, 10, 123, 456`
- Floating point constants: `12.34, 0.5, 100.0`

6. Identifiers

Identifiers follow the rule:

```
letter (letter | digit)*
```

Tokens to be Recognized

Token Class	Examples
KEYWORD	if, int, while
ID	sum, total
NUM_INT	123
NUM_FLOAT	12.34
OPERATOR	+, -, *, /, =
RELOP	<, <=, >, >=, ==, ! =
DELIMITER	; , ( ) { }
INVALID	@, #, $

What Exactly the Student Has To Do

Each student must complete the following tasks carefully and systematically.

Task 1: Study the Language Specification

1. Carefully study the given alphabet set, keywords, operators, relational operators, constants, and identifier rules.
2. From this specification, prepare a list of all **token classes** that your lexical analyzer must recognize.

Task 2: Design Transition Diagrams (To be drawn in Lab Record)

Draw **separate transition diagrams** for each of the following token categories:

1. Identifier / Keyword
2. Integer constant
3. Floating point constant
4. Relational operators (`<, <=, >, >=, ==, !=`)
5. Operators (`+, -, *, /, =`)

Each diagram must clearly show:

- Start state
- Accepting state(s)
- Transitions with proper labels

These diagrams must be **neatly drawn and submitted in the laboratory record**.

Task 3: Implement the Lexical Analyzer

Write a program in **C** that performs the following:

1. Read a source program from an input file (e.g., `input.toy`).
2. Scan the input file **character by character**.
3. Skip whitespace, tabs, and newlines.
4. Use the logic of the designed transition diagrams to:
   - Recognize tokens
   - Extract lexemes
   - Classify each token correctly
5. For each recognized token, print the output in the following format:

   `<TOKEN_CLASS, LEXEME>`

Task 4: Implement and Maintain the Symbol Table

Your program must:

1. Create a symbol table data structure.
2. Whenever an **identifier** is encountered:
   - Check whether it already exists in the symbol table.
   - If it does not exist, insert it into the symbol table.
3. Do **not** insert:
   - Keywords
   - Operators
   - Constants
4. At the end of execution, print the symbol table in a **tabular form**.

Task 5: Test Using the Given Sample Program

1. Store the given sample program in a file (e.g., `input.toy`).
2. Run your lexical analyzer on this file.
3. Verify that:
   - All valid tokens are correctly identified.
   - No valid token is missed.
   - No incorrect token is generated.
4. Include the program output in your laboratory record.

Task 6: Demonstration and Viva

During laboratory evaluation, each student must:

1. Show:
   - The transition diagrams
   - The source code
   - The output of the lexical analyzer
   - The generated symbol table
2. Explain:
   - How tokens are recognized
   - How the symbol table is updated
   - How invalid tokens or errors are handled

Sample Input Program

```
begin
    int a, b;
    float sum;
    a = 10;
    b = 20;
    sum = a + b;
    if (a < b)
        print sum;
end
```

Symbol Table Format (Example)

Index	Name	Type
1	a	ID
2	b	ID
3	sum	ID

Appendix: Useful C Library Functions for This Assignment

Students are allowed to use the following standard C library functions for implementing the lexical analyzer.

1. File Handling Functions (`stdio.h`)

Function	Syntax	Purpose
fopen	FILE *fopen(const char *fname, const char *mode);	Opens a file for reading or writing.
fclose	int fclose(FILE *fp);	Closes an open file.
fgetc	int fgetc(FILE *fp);	Reads one character from a file.
ungetc	int ungetc(int ch, FILE *fp);	Pushes one character back to the input stream (used for lookahead).
feof	int feof(FILE *fp);	Checks whether end-of-file has been reached.
printf	int printf(const char *format, ...);	Prints output to the screen.

2. Character Classification Functions (ctype.h)

Function	Syntax	Purpose
isalpha	int isalpha(int ch);	Checks if the character is a letter (A–Z or a–z).
isdigit	int isdigit(int ch);	Checks if the character is a digit (0–9).
isalnum	int isalnum(int ch);	Checks if the character is a letter or digit.
isspace	int isspace(int ch);	Checks if the character is a whitespace (space, tab, newline, etc.).

3. String Handling Functions (string.h)

Function	Syntax	Purpose
strcpy	char *strcpy(char *dest, const char *src);	Copies one string into another.
strcmp	int strcmp(const char *s1, const char *s2);	Compares two strings. Returns 0 if they are equal.
strlen	size_t strlen(const char *s);	Returns the length of a string.

4. General Utility Functions (stdlib.h)

Function	Syntax	Purpose
exit	void exit(int status);	Terminates the program immediately.
malloc	void *malloc(size_t size);	Allocates memory dynamically (optional for symbol table).
free	void free(void *ptr);	Frees dynamically allocated memory.

Important Note

Students are encouraged to use only the above standard library functions and must implement the token recognition logic, symbol table management, and transition diagram simulation **on their own**.

Appendix: Reference Implementation of a Dynamic Symbol Table (C)

Library Files:

```c
/* Required C header files */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

The following C code shows a clean and simple implementation of a **dynamic, table-like symbol table** using a resizable array. This implementation supports:

- Creation of a dynamic symbol table
- Lookup of symbols
- Insertion of new symbols
- Printing of the symbol table
- Proper memory deallocation

Data Structure Definition

```c
/* ======================
   Symbol Table Entry
   ====================== */
typedef struct {
    char name[64];      /* Identifier name */
    char type[32];      /* For this lab: "ID" */
    int line;           /* Line number where first seen */
} Symbol;

/* ======================
   Dynamic Symbol Table
   ====================== */
Symbol *symbolTable = NULL;   /* Dynamic array */
int symbolCount = 0;          /* Number of entries used */
int symbolCapacity = 0;       /* Current allocated size */
```

Lookup Function

```c
/* Returns index if found, otherwise returns -1 */
int lookupSymbol(const char *name)
{
    for (int i = 0; i < symbolCount; i++) {
        if (strcmp(symbolTable[i].name, name) == 0)
            return i;
    }
    return -1;  /* Not found */
}
```

Insertion Function (With Automatic Resizing)

```c
void insertSymbol(const char *name, const char *type, int line)
{
    /* Do not insert duplicates */
    if (lookupSymbol(name) != -1)
        return;

    /* Grow table if needed */
    if (symbolCount == symbolCapacity) {
        int newCapacity = (symbolCapacity == 0) ? 10 :
           symbolCapacity * 2;

        Symbol *newTable = (Symbol *)realloc(symbolTable,
           newCapacity * sizeof(Symbol));
        if (newTable == NULL) {
            printf("Error: Memory allocation failed for symbol
               table.\n");
            exit(1);
        }

        symbolTable = newTable;
        symbolCapacity = newCapacity;
    }

    /* Insert new entry */
    strcpy(symbolTable[symbolCount].name, name);
    strcpy(symbolTable[symbolCount].type, type);
    symbolTable[symbolCount].line = line;

    symbolCount++;
}
```

Printing the Symbol Table

```c
void printSymbolTable(void)
{
    printf("\n================ SYMBOL TABLE ================\n");
```

```
    printf("%-5s␣%-20s␣%-10s␣%-10s\n", "No.", "Name", "Type", "Line
       ");
    printf("-----------------------------------------\n");

    for (int i = 0; i < symbolCount; i++) {
        printf("%-5d␣%-20s␣%-10s␣%-10d\n",
               i + 1,
               symbolTable[i].name,
               symbolTable[i].type,
               symbolTable[i].line);
    }
}
```

Freeing the Symbol Table Memory

```
void freeSymbolTable(void)
{
    free(symbolTable);
    symbolTable = NULL;
    symbolCount = 0;
    symbolCapacity = 0;
}
```

How to Use in the Lexical Analyzer

```
/* When an identifier is recognized */
insertSymbol(lexeme, "ID", currentLine);

/* At the end of main() */
printSymbolTable();
freeSymbolTable();
```

Note

This implementation uses a dynamically growing array and the `realloc()` function to automatically increase the size of the symbol table when needed.